MVC ARCHITECTURE

Model · View · Controller — with a Relational Database

A hands-on lecture built around the Book Library project (Node.js · Express · EJS · SQLite)

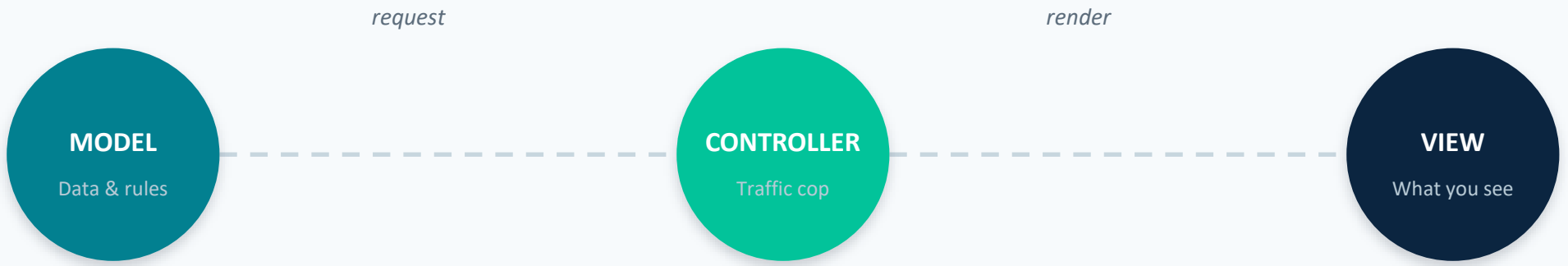
Learning Objectives

By the end of this lecture, you will be able to:

1. Explain the Model–View–Controller pattern and why it separates concerns
2. Trace a request end-to-end through routes, controller, model, and view
3. Read and write code that queries a relational database from a model layer
4. Run, extend, and debug a small full-stack MVC application in VS Code

What Is MVC?

A design pattern that splits an application into three interconnected parts, each with one job.



Key idea: the Controller never touches the database directly, and the View never contains business logic. Each layer can change without breaking the others.

Why Separate Concerns?

Maintainability

Fix a bug in one layer without touching the others

Reusability

The same Model can power a web view, an API, or a CLI

Team workflow

Front-end and back-end developers can work in parallel

Testability

Business logic in Model/Controller is easy to test without a browser

The Three Layers, Defined

MODEL

Owns the data.

Talks to the database. Contains queries, validation rules, and business logic. Knows nothing about HTTP or HTML.

In this project:

`models/bookModel.js`

VIEW

Owns the display.

Renders data as HTML the browser can show. Contains no database calls and as little logic as possible.

In this project:

`views/*.ejs`

CONTROLLER

Owns the flow.

Receives the HTTP request, calls the Model, decides what happens next, and picks a View to render.

In this project:

`controllers/bookController.js`

Case Study: The Book Library App

A small CRUD app (Create, Read, Update, Delete) for managing a list of books — used throughout this lecture as our running example.

Express.js

CONTROLLER

Handles HTTP requests & routes

EJS

VIEW

Renders HTML templates

SQLite

MODEL

Relational database, single file

Node.js

RUNTIME

Runs the whole application

Relational database: books are stored as rows in a single "books" table — the same relational model used by MySQL, PostgreSQL, or SQL Server.

Project Structure

```
mvc-book-library/
|-- app.js                  entry point
|-- config/
|   `-- db.js               DB connection + setup
|-- models/
|   `-- bookModel.js        MODEL  (all SQL queries)
|-- controllers/
|   `-- bookController.js   CONTROLLER (request logic)
|-- routes/
|   `-- bookRoutes.js       URL -> controller mapping
|-- views/
|   |-- index.ejs             VIEW (list)
|   |-- add.ejs               VIEW (add form)
|   `-- edit.ejs              VIEW (edit form)
`-- database/library.db     SQLite data file
```

Why organize it this way?

- One folder per layer keeps responsibilities obvious at a glance.
- routes/ is thin — it only maps URLs to controller functions.
- controllers/ never contains SQL — it always calls models/.
- views/ never queries the database directly.
- config/ isolates environment-specific setup (the DB connection).

The Model Layer

models/bookModel.js — the only file that writes SQL

```
const db = require('../config/db');

const BookModel = {
  getAll: (callback) => {
    db.all('SELECT * FROM books ORDER BY id DESC',
      [], callback);
  },

  create: (book, callback) => {
    const { title, author, genre,
            published_year } = book;
    db.run(
      `INSERT INTO books
       (title, author, genre, published_year)
       VALUES (?, ?, ?, ?)`,
      [title, author, genre, published_year],
      callback
    );
  },
};

module.exports = BookModel;
```

Notice

Uses parameterized queries (?) — never string-concatenates user input into SQL. This prevents SQL injection.

Every function takes a callback — the Controller decides what to do with the result or error.

No req, res, or HTML anywhere in this file.

The View Layer

views/index.ejs — turns data into HTML

```
<h1><%= title %></h1>

<% if (books.length === 0) { %>
  <p>No books yet.</p>
<% } else { %>
  <table>
    <% books.forEach(book => { %>
      <tr>
        <td><%= book.title %></td>
        <td><%= book.author %></td>
        <td>
          <a href="/books/<%= book.id %>/edit">
            Edit
          </a>
        </td>
      </tr>
    <% }) %>
  </table>
<% } %>
```

Notice

<%= %> outputs data the Controller passed in — the view never fetches its own data.

<% %> runs plain JS for loops/conditionals — but no database calls belong here.

The same pattern (loop + interpolate) works for add.ejs and edit.ejs forms.

C The Controller Layer

controllers/bookController.js — the traffic cop

```
const BookModel = require('../models/bookModel');

create: (req, res) => {
  const { title, author, genre,
          published_year } = req.body;

  if (!title || !author) {
    return res.status(400)
      .send('Title and Author required.');
  }

  BookModel.create(
    { title, author, genre, published_year },
    (err) => {
      if (err) return res.status(500)
        .send('DB error: ' + err.message);
      res.redirect('/');
    }
  );
}
```

Notice

Reads req.body (from the View's form) and validates it before touching the Model.

Delegates the actual database work to BookModel.create — never writes SQL itself.

Decides the outcome: redirect on success, an error response on failure.

Routes: Connecting URLs to Controllers

routes/bookRoutes.js maps an HTTP method + URL pattern to exactly one controller function.

Method	URL	Controller Function	Purpose
GET	/	`index`	List all books
GET	/books/add	`showAddForm`	Show the add-book form
POST	/books	`create`	Insert a new book
GET	/books/:id/edit	`showEditForm`	Show a pre-filled edit form
PUT	/books/:id	`update`	Update an existing book
DELETE	/books/:id	`destroy`	Remove a book

Request Lifecycle: Adding a Book

Follow one request from the browser all the way to the database and back.

1	User submits the Add Book form	`VIEW → POST /books`
2	Route maps the request	`ROUTES → BookController.create`
3	Controller validates input	`CONTROLLER → calls BookModel.create()`
4	Model runs the SQL insert	`MODEL → INSERT INTO books ...`
5	Controller redirects to “/”, page re-renders with the new book	`CONTROLLER → VIEW`

The Relational Database

SQLite stores data as rows in tables with a fixed schema — the same relational model as MySQL or PostgreSQL.

books table

Column	Type	Notes
id	INTEGER	Primary key, auto-increment
title	TEXT	Required
author	TEXT	Required
genre	TEXT	Optional
published_year	INTEGER	Optional

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY
     AUTOINCREMENT,
  title TEXT NOT NULL,
  author TEXT NOT NULL,
  genre TEXT,
  published_year INTEGER
);
```

Try it yourself: add an authors table with its own id, then give books an author_id foreign key. Update bookModel.js to JOIN the two tables.

Running the App in VS Code

1. Open the mvc-book-library folder in VS Code
2. Open a terminal: Terminal → New Terminal
3. Install dependencies: npm install
4. Start the server: npm start
5. Visit http://localhost:3000 in your browser

```
$ npm install

$ npm start

Server running at
http://localhost:3000
```

The SQLite file database/library.db is created automatically on first run — no separate DB server to install.

Practice Exercises

Extend the Book Library app to deepen your understanding of each layer.

Relational join

Add an authors table with a foreign key from books.author_id

Validation

Show inline error messages in the View when title/author are missing

Search

Add GET /?q=... using a WHERE title LIKE ? query in the Model

New field

Add an is_read column and a route to toggle it (PATCH)

Summary

- MVC separates an app into Model (data), View (display), and Controller (flow)
- The Controller is the only layer that talks to both the Model and the View
- A relational database organizes data into tables with fixed columns and typed rows
- The same pattern scales from a 4-file demo to large production applications

Questions?

Project files: mvc-book-library.zip • README.md has setup + extension ideas